

If constructors can be overloaded to support multiple ways of object creation, can methods also support multiple ways of execution?

Example

```
class calculator{  
  
    public int add(int n1, int n2){  
        return n1+n2;  
    }  
class Main {  
  
    public static void main(String[] args) {  
        calculator obj = new calculator();  
        int r = obj.add(5,7);  
        System.out.println(r);  
    }  
}
```

What if we want to
add 3 numbers
instead of two?

Solution 01

```
class calculator{
    public int add_two(int n1, int n2){
        return n1+n2;
    }
    public int add_three(int n1, int n2, int n3){
        return n1+n2+n3;
    }
}

class Main {
    public static void main(String[] args) {
        calculator obj = new calculator();
        int r2 = obj.add_two(5,7);
        int r3 = obj.add_three(5,7,9);
        System.out.println(r2);
        System.out.println(r3);
    }
}
```

Solution 02 (Overloading)

```
class calculator{
    public int add(int n1, int n2){
        return n1+n2;
    }
    public int add(int n1, int n2, int n3){
        return n1+n2+n3;
    }
}
class Main {
    public static void main(String[] args) {
        calculator obj = new calculator();
        int r2 = obj.add(5,7);
        int r3 = obj.add(5,7,10);
        System.out.println(r2);
        System.out.println(r3);
    }
}
```

Method Overloading

- ◆ Two or more methods in same class having same:
 - ◆ name
 - ◆ return type
- ◆ Having different:
 - ◆ Different type
 - ◆ Different number
 - ◆ Different Sequence of arguments
- ◆ Type or number of argument determine the method to be called

Different Number of Arguments

```
class calculator{
    public int add(int n1, int n2){
        return n1+n2;

    }
    public int add(int n1, int n2, int n3){
        return n1+n2+n3;

    }
}
class Main {
    public static void main(String[] args) {
        calculator obj = new calculator();

        int r2 = obj.add(5,7);
        int r3 = obj.add(5,7,10);

        System.out.println(r2);
        System.out.println(r3);
    }
}
```

Different Types

```
class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    // same method name but different parameter types  
    public double add(double a, double b) {  
        return a + b;  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        Calculator obj = new Calculator();  
  
        int result1 = obj.add(5, 7);    // calls int version  
        double result2 = obj.add(5.5, 7.2); // calls double version  
  
        System.out.println(result1);  
        System.out.println(result2);  
    }  
}
```


Different Sequence

```
class Calculator {  
  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    // different sequence of parameters  
    public int add(int a, double b) {  
        return (int)(a + b);  
    }  
  
    public int add(double a, int b) {  
        return (int)(a + b);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
  
        Calculator obj = new Calculator();  
  
        System.out.println(obj.add(5, 7));    // int, int  
        System.out.println(obj.add(5, 7.5)); // int, double  
        System.out.println(obj.add(7.5, 5)); // double, int  
    }  
}
```

Method Overloading Benefits

- ◆ Advantages:
 - ◆ Improve code readability
 - ◆ Reuse the same method name
 - ◆ Perform similar operations with different inputs
- ◆ Those languages which don't support overloading(like C):
 - ◆ Give unique names for methods
 - ◆ `abs()` for `int` and `labs()` for `long int`
- ◆ It is one of the ways in which java can achieve polymorphism:
 - ◆ Polymorphism: One object shows multiple behaviors

Real-life Example 01

- ◇ Printing Service: Think of a **printer system** in an office.
- ◇ We can print:
 - ◇ a **text document**
 - ◇ an **image**
 - ◇ a **PDF file**
- ◇ The action is still **print**, but the **input type changes**.

Real-life Example 02

- ◇ Payment System
- ◇ In an **e-commerce system**, the action **pay()** is the same, but the payment method changes.
 - ◇ `pay(CreditCard card)`
 - ◇ `pay(DebitCard card)`
 - ◇ `pay(UPI upi)`
- ◇ So the system chooses the correct **pay() method** depending on the **type of payment object**.

Real-life Example 03

- ◆ In **Google Maps**, you can search using:
 - ◆ a **place name**: `search(String placeName)`
 - ◆ **Coordinates**: `search(double latitude, double longitude)`
 - ◆ a **postal code**: `search(int postalCode)`
- ◆ The action is still **search()**, but the input type differs.

Constructor vs Method Overloading

Feature	Constructor Overloading	Method Overloading
Purpose	Initialize object	Perform operations
Name	Same as class	Any method name
Return Type	No return type	Can have return type

If an object contains multiple variables, should we pass each variable separately, or can we pass the whole object to a method?

Passing Data to Methods

- ◈ Previously, we passed **primitive values** (like int, double, char) to methods.
- ◈ Here we passed an **integer value** to the method.
- ◈ But sometimes we need to **pass an entire object to a method**.

```
class Test {  
  
    void display(int num) {  
        System.out.println(num);  
    }  
  
    public static void main(String[] args) {  
        Test obj = new Test();  
        obj.display(10);  
    }  
}
```


What Happens Internally

- ◇ When an object is passed to a method.
 - ◇ The **reference of the object** is passed
 - ◇ The method can **access the object's variables**
- ◇ The method **modified the original object**.

Why Pass Objects to Methods?

- ◇ Objects usually contain **multiple variables and behaviors**.
- ◇ Instead of passing many variables separately, we can **pass the object itself**.
- ◇ This allows a method to **access the object's data directly**.
- ◇ Example idea:
 - ◇ A **Student object** contains name and age
 - ◇ A method may need the **entire student object**

Example: Passing an Object

```
class Student {  
    String name;  
    int age;  
}  
  
class Test {  
  
    void display(Student s) {  
        System.out.println(s.name + " " + s.age);  
    }  
  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
        s1.name = "Ali";  
        s1.age = 20;  
  
        Test obj = new Test();  
        obj.display(s1);  
    }  
}
```

Applications

- ◇ Access object data
- ◇ **Modify object values**
- ◇ Perform calculations using object attributes
- ◇ **Compare objects**
- ◇ **Share objects between classes (Homework)**

Scenario 1 — Displaying Object Information

```
class Student {  
    String name;  
    int age;  
  
    void display(Student s) {  
        System.out.println(s.name + " " + s.age);  
    }  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
        s1.name = "Ali";  
        s1.age = 20;  
  
        Student obj = new Student();  
        obj.display(s1);  
    }  
}
```

- ◈ When a method needs to **read and display object data**.
- ◈ Here the **Student object is passed to the method** to display its data.

Scenario 2 — Modifying Object Data

```
class Student {  
    int age;  
  
    void increaseAge(Student s) {  
        s.age = s.age + 1;  
    }  
  
    public static void main(String[] args)  
    {  
  
        Student s1 = new Student();  
        s1.age = 20;  
  
        Student obj = new Student();  
        obj.increaseAge(s1);  
  
        System.out.println(s1.age);  
    }  
}
```

- ◆ A method can **update or modify** the object's **values**.
- ◆ The method **modifies the original object**.

Scenario 3 — Performing Operations on Objects

```
class Rectangle {  
    int length;  
    int width;  
  
    int area(Rectangle r) {  
        return r.length * r.width;  
    }  
  
    public static void main(String[] args) {  
  
        Rectangle rect = new Rectangle();  
        rect.length = 10;  
        rect.width = 5;  
  
        Rectangle obj = new Rectangle();  
        System.out.println(obj.area(rect));  
    }  
}
```

- ◆ When a method needs to **perform calculations using object data**.
- ◆ The method uses the **object's attributes for computation**.

Scenario 4 — Comparing Two Objects

```
class Student {  
    int age;  
  
    void compare(Student s1, Student s2) {  
  
        if (s1.age > s2.age)  
            System.out.println("First student is older");  
        else  
            System.out.println("Second student is  
older");  
    }  
  
    public static void main(String[] args) {  
  
        Student a = new Student();  
        Student b = new Student();  
  
        a.age = 22;  
        b.age = 20;  
  
        Student obj = new Student();  
        obj.compare(a, b);  
    }  
}
```

- ◆ Methods can receive **objects to compare their data**.
- ◆ The method compares **data stored in objects**.

Summary

- Sharing complete objects
- Accessing multiple variables
- Modifying the original object's data
- This is commonly used in real-world Java programs and object-oriented design.

What if we don't
want other classes to
directly change our
object's data? How
can we protect it?

Importance of Making Data Private

- ◆ To prevent incorrect or unauthorized modification, we **restrict direct access to variables**.
- ◆ Java allows us to do this using the **private access modifier**.

Example:

```
</> Java  
  
class Student {  
    private int age;  
}
```

Now the variable **cannot be** accessed directly from outside the class.

Example:

```
</> Java  
  
Student s1 = new Student();  
s1.age = 20;    // Not allowed
```

This helps **protect the internal data** of the object.

Access Modifiers

Private: Accessible **only within the class**.
Not accessible outside the class.

Default (no modifier): Accessible **within the package** only. Not accessible outside the package.

Protected: Accessible **within the package** and **in subclasses outside the package**.

Public: Accessible **everywhere**—within the class, package, and from any other class.

Access Modifiers

Access Modifier	Within Class	Within Package	Subclass (Outside Package)	Everywhere
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

How Can We Access Private Data Then?

- ◇ If variables are **private**, we still need a **controlled way to read or modify them**.
- ◇ We do this using **public methods**.
- ◇ Two common methods are:
 - ◇ **Setter method** → to modify data
 - ◇ **Getter method** → to read data

Example 01

```
class Student {  
  
    private int age;  
  
    public void setAge(int a) {  
        age = a;  
    }  
  
    public int getAge() {  
        return age;  
    }  
}  
class Main {  
    public static void main(String[] args) {  
  
        Student s1 = new Student();  
  
        s1.setAge(20);           // setting value  
        System.out.println(s1.getAge()); // accessing value  
    }  
}
```

Example 02

```
class Test{
    private int data;

    void setData(int d){
        data = d;
    }

    int getData(){
        return data;
    }
}

class Main {

    public static void main(String[] args) {
        Test obj = new Test();
        obj.setData(25);
        System.out.println(obj.getData());
    }
}
```

What if we don't or forgot to initialize data using set function?

With Default constructor

```
class Test{
    private int data;

    Test () {
        data = 20;
    }

    void setData(int d){
        data = d;
    }

    int getData(){
        return data;
    }
}

class Main {

    public static void main(String[] args) {
        Test obj = new Test();
        //obj.setData(25);
        System.out.println(obj.getData());
    }
}
```

With Default and Parameterized constructor

```
class Test{
    private int data;

    Test () {
        data = 20;
    }

    Test (int k){
        data = k;
    }

    void setData(int d){
        data = d;
    }

    int getData(){
        return data;
    }
}

class Main {

    public static void main(String[] args) {
        Test obj = new Test(30);
        //obj.setData(25);
        System.out.println(obj.getData());
    }
}
```

Encapsulation

- ◆ It is simple concept of:
 - ◆ **Making data private**
 - ◆ **Providing public methods to access it**
- ◆ Encapsulation is the process of binding data and methods together in a class
- ◆ While restricting direct access to the data.

So instead of:

⌞ Java

```
object.variable
```

we use:

⌞ Java

```
object.setVariable()
```

```
object.getVariable()
```

Key Idea

Is Encapsulation just about making data private and giving access through public methods?

Quick Questions

Can someone set a student's age to -10 even with getters/setters?

Can a bank account balance be negative without rules?

What Happens Without Encapsulation?

```
class BankAccount {  
    public double balance;  
}
```

```
class Main {  
    public static void main(String[] args) {  
        BankAccount acc = new  
BankAccount();  
        acc.balance = -5000;  
        System.out.println(acc.balance);  
    }  
}
```

Problems Without Encapsulation?

- ❖ Invalid values can be stored
- ❖ Program logic can break
- ❖ Sensitive data can be modified incorrectly
- ❖ No control over how data is updated
- ❖ Here the program allows an impossible age.

```
class Student {  
    public int age;  
}  
  
class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
  
        s1.age = -10;  
        System.out.println(s1.age);  
    }  
}
```

The Solution: Enforce Rules to access data

```
class BankAccount {  
  
    private double balance;  
  
    public void setBalance(double b) {  
        if (b >= 0) {  
            balance = b;  
        } else {  
            System.out.println("Invalid balance");  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

Using the Controlled Access

```
class BankAccount {  
  
    private double balance;  
  
    public void setBalance(double b) {  
        if (b >= 0) {  
            balance = b;  
        } else {  
            System.out.println("Invalid balance");  
        }  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        BankAccount acc = new BankAccount();  
  
        acc.setBalance(5000);  
        System.out.println(acc.getBalance());  
  
        acc.setBalance(-2000);  
    }  
}
```



Encapsulation = data hiding + controlled access + rules
for safe/valid operations.

Application 01: Banking Systems

Account balances and transactions are **private**.

Updates occur only through controlled methods (deposit(), withdraw()).

Ensures **data integrity and security**.

Application 01: Banking Systems

```
class BankAccount {  
    private double balance;  
  
    public void deposit(double amount) {  
        if(amount > 0)  
            balance += amount;  
    }  
  
    public void withdraw(double amount) {  
        if(amount > 0 && amount <= balance)  
            balance -= amount;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        BankAccount acc = new BankAccount();  
        acc.deposit(5000);  
        acc.withdraw(2000);  
        System.out.println(acc.getBalance()); // 3000  
    }  
}
```

Application 02: E-Commerce Platforms

Inventory levels, pricing, and discounts are **encapsulated**.

Methods enforce **business rules** before updating values.

Guarantees **consistent stock management and accurate billing**.

Application 02: E- Commerce Platforms

```
class Product {  
    private int stock;  
    private double price;  
  
    public void updateStock(int quantity) {  
        if(quantity >= 0)  
            stock = quantity;  
    }  
  
    public void updatePrice(double p) {  
        if(p >= 0)  
            price = p;  
    }  
  
    public void display() {  
        System.out.println("Stock: " + stock + ", Price: " + price);  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        Product p = new Product();  
        p.updateStock(50);  
        p.updatePrice(999.99);  
        p.display();  
    }  
}
```

Application 03: Robotics and IoT Devices

Sensor readings and actuator controls are **private**.

Methods validate input and enforce **safety limits**.

Protects **hardware reliability and operational safety**.

Application 03: Robotics and IoT Devices

```
class Motor {
    private int speed;

    public void setSpeed(int s) {
        if(s >= 0 && s <= 100)
            speed = s; // safe limit
    }

    public int getSpeed() {
        return speed;
    }
}

class Main {
    public static void main(String[] args) {
        Motor m = new Motor();
        m.setSpeed(80);
        System.out.println("Motor speed: " + m.getSpeed());
        m.setSpeed(150); // ignored due to rule
        System.out.println("Motor speed: " + m.getSpeed());
    }
}
```

Summary

- ◊ Without encapsulation:
 - ◊ Anyone can modify the data
 - ◊ Incorrect values can enter the system
- ◊ With encapsulation:
 - ◊ Data is **protected**
 - ◊ Access is **controlled**
 - ◊ Rules can be **applied before updating values**
- ◊ This approach ensures **safer and more reliable programs**.